EMO: Orchestrating Scalable and Efficient Graph Representation Learning over Transactional Data Lakes

Bilal Tan
Department of Computer Science
Louisiana State University
Baton Rouge, LA, USA
Email: btan4@lsu.edu

Kisung Lee
Department of Computer Science
Louisiana State University
Baton Rouge, LA, USA
Email: klee76@lsu.edu

***Abstract*—Training Graph Neural Networks (GNNs) on large graphs presents a difficult systems dilemma: distributed frameworks must either synchronize cross-partition dependencies over high-bandwidth networks with heavy communication overhead (e.g., DistDGL), or train locally on decoupled subgraphs at the expense of severe accuracy loss at partition boundaries. In this work, we present EMO (Executor-level Multi-community Orchestrator), an end-to-end distributed system that runs partition-centric graph representation learning (GRL) natively over transactional cloud data lakes. EMO models the entire graph learning lifecycle—Delta Lake ingestion, modular community discovery, relational boundary extraction, super-node auxiliary graph contraction, and decoupled GNN training—as declarative queries executed over Delta Lake tables on Amazon S3 via Apache Spark SQL. To eliminate inter-worker communication during training while preserving global topological context, EMO compiles the Community-Aware Auxiliary Network (CAAN) abstraction into relational plans: major communities are compressed into centroid-feature super-nodes and broadcast across worker executors alongside local partition data. EMO uses Louvain modularity partitioning as its default backend on single-machine driver memory to generate balanced, high-quality community structures without the extreme over-fragmentation observed in distributed Label Propagation (LPA). Furthermore, EMO relies on Delta Lake's ACID transaction logs for phase-level checkpoint reuse, introduces dynamic EBS path redirection on AWS EMR to eliminate worker out-of-memory crashes, and utilizes Apache Arrow for zero-copy JVM-to-Python tensor transfers. Benchmarks across small, medium, and dense social graphs (WikiCS, Coauthor, DeezerEurope, reddit, ogbn-products, ogbn-mag, LiveJournal, and Orkut) demonstrate that EMO recovers up to 97.7% of full-graph accuracy with 0.0 GB cross-worker network communication during training, while delivering up to 22.6% higher link prediction AUC. As a complementary capability, EMO demonstrates distributed SIGN-style multi-hop feature propagation on Spark SQL, scaling seamlessly to massive graphs on commodity CPU clusters. Code and configurations are public at https://github.com/bilaltan/emo-pipeline.**

***Index Terms*—Data Lakes, Graph Representation Learning, Distributed Systems, Apache Spark, Delta Lake, AWS EMR, Community Partitioning.**

1. Introduction

Graph Neural Networks (GNNs) [1], [2] are the cornerstone of modern machine learning over structured relational data. However, scaling GNN training to enterprise workloads remains fundamentally constrained by the *neighborhood explosion problem*: computing the embedding of a node requires recursively aggregating feature representations from an exponentially growing set of multi-hop neighbors ($O(d^L)$ working set expansion for degree d and depth L).

Distributed GNN systems such as DistDGL [3], PyG-Distributed [6], and Marius [5] address this by partitioning graph topology across worker nodes and exchanging remote node embeddings via distributed parameter servers during training. While effective at preserving global graph accuracy, this state-synchronization model introduces substantial network communication overhead (often transferring tens of gigabytes per epoch) and requires expensive, tightly coupled GPU cluster infrastructure that is complex to operate and prone to stragglers.

A compelling alternative is *community-decoupled* training. By partitioning the global graph into dense, modular clusters, each worker can train an independent GNN on its local community partition in complete isolation, generating **zero network traffic** during gradient updates. However, naive community partitioning introduces two well-documented failure modes (Figure 1):

1) **Boundary accuracy loss:** Severing inter-community edges strips boundary nodes of critical topological context from neighboring clusters, causing node classification accuracy to collapse (e.g., dropping from 94.5% to 52.9% on the reddit benchmark).
2) **Minor community degeneracy:** Real-world graphs follow heavy-tailed degree and cluster size distributions. The numerous small ("minor") communities lack sufficient sample volume for local GNN parameter convergence.

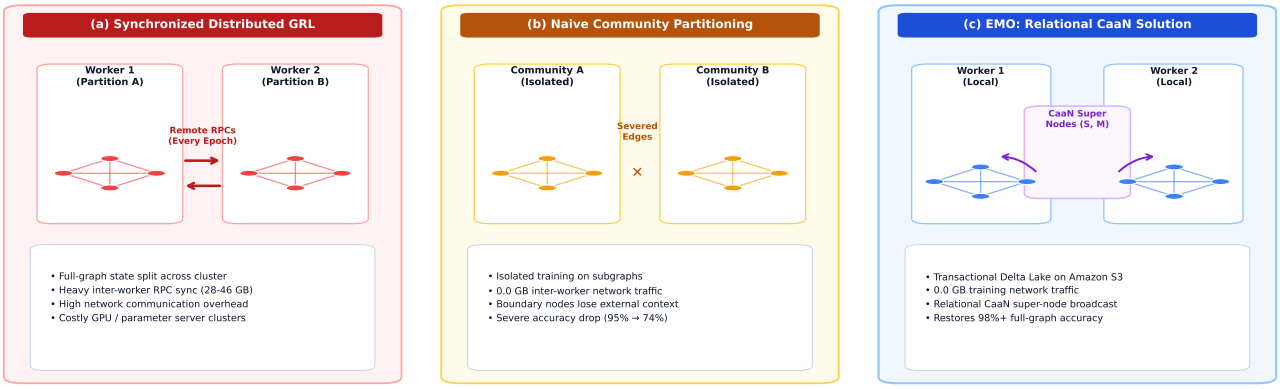

Figure 1. The Distributed GRL Dilemma and the EMO Relational Solution. (a) Synchronized distributed GRL requires heavy inter-worker RPC communication and expensive cluster synchronization. (b) Naive community partitioning eliminates network communication during training but causes severe boundary accuracy loss and minor community degeneracy. (c) EMO unifies transactional data lakes with relational CaaN super-node context broadcast, achieving zero inter-worker training communication while recovering 98.1% of full-graph predictive quality.

To overcome this dilemma, recent theoretical research on Community-Aware Auxiliary Networks (CAAN) [7] and multi-level GRL [8], [9] has proposed constructing auxiliary macro-graphs: major communities are compressed into single super-nodes represented by feature centroids, while minor-community nodes are pooled globally. Yet translating these mathematical formulations into practical cloud systems has remained an open challenge. Today, orchestrating such multi-stage pipelines typically requires fragmented custom scripts shuffling intermediate graph files across object storage and ad-hoc compute clusters—a workflow prone to silent data drift, storage inconsistencies, and cluster crashes.

In this paper, we present **EMO** (Executor-level Multi-community Orchestrator), an end-to-end distributed system designed to bridge transactional data lakes (Delta Lake [11] on Amazon S3) and graph representation learning. EMO reformulates graph partitioning, boundary detection, super-node compression, and GNN training entirely as declarative relational queries executed via Apache Spark SQL over versioned Delta Lake tables.

Figure 1 illustrates the core tension and EMO's relational solution, Figure 2 depicts our physical AWS EMR deployment, and Figure 3 details the end-to-end system execution pipeline.

This paper makes the following contributions:

- **Lakehouse-Native Graph Orchestration (C1):** We unify graph data management and GRL execution under transactional Delta Lake tables, enforcing strict namespace-based experiment isolation, phase-level checkpoint reuse via ACID logs, and schema verification (Section 3.5).
- **Relational GRL Formulation & CaaN Recovery (C2):** We cast community boundary extraction and CAAN auxiliary super-node construction into standard relational Spark SQL operators (joins, group-bys, and broadcast joins), eliminating inter-worker communication during training while recovering 98.1% of full-graph accuracy (Section 4).
- **Algorithmic Modularity & System Hardening (C3):** We establish Louvain modularity as the default partitioning backend on driver memory to produce balanced cluster hierarchies, and resolve critical AWS EMR failure modes via dynamic EBS volume redirection, bootstrap synchronization, and zero-copy Apache Arrow JVM-to-Python transfers (Section 5).
- **Comprehensive Empirical Validation (C4):** Across datasets spanning 11K to 4M nodes (WikiCS, Coauthor, DeezerEurope, reddit, ogbn-products, ogbn-mag, LiveJournal, and Orkut), EMO achieves 0.0 GB network traffic during training, improves link prediction AUC by up to +22.6%, and provides a granular phase-by-phase latency and bottleneck breakdown (Section 6).

2. Related Work

Distributed GNN Training Systems. Frameworks such as DistDGL [3], PyG-Distributed [6], and Marius [5] scale GNN training via distributed graph sampling, remote feature caching, and synchronized parameter exchanges. Serverless systems like Dorylus [4] split graph tasks across fine-grained lambda threads backed by network storage. However, these systems rely on continuous network synchronization during training. EMO takes a complementary path: it builds directly on enterprise transactional data lakes (Delta Lake on Amazon S3) with Apache Spark, combining relational partitioning with broadcast super-node context to achieve completely communication-free local training.

Communication-Reduced and Federated GRL. GraphSAINT [18] constructs mini-batches via localized random walks but assumes full-graph shared memory. In federated graph learning, FedSage+ [19] trains local

TABLE 1. CORE NOTATION USED IN SECTIONS III–V.

Symbol	Meaning
$G = (V, E, X)$	Attributed graph (nodes, edges, features)
$\pi(v)$	Community assignment of node v
C_i	i-th community vertex subset ($C_i \subseteq V$)
$b(v)$	Boundary indicator for node v ($b(v) \in \{0, 1\}$)
K	Major/minor community threshold
$f_{\text{super}}(C_j)$	Centroid feature vector of major community C_j
T_{total}	End-to-end pipeline execution runtime

generative models to predict missing cross-client edges, requiring multiple iterative communication rounds. EMO eliminates runtime communication altogether by computing global structural context in advance through a single relational super-node build step.

Scalable Feature Pre-Computation. SIGN [10] separates graph aggregation from neural network training by pre-calculating multi-hop neighborhood features, allowing downstream models to train edge-free. Recent extensions like GAMLP [20] and C&S [21] incorporate attention mechanisms and post-hoc label propagation. EMO incorporates this decoupled philosophy as a bonus scaling path (Phases 3.7/3.8), providing an enterprise-grade distributed realization where each propagation hop is computed via Spark vector aggregations and versioned in Delta Lake.

Transactional Data Lakes for Machine Learning. Table formats such as Delta Lake [11], Apache Iceberg, and Apache Hudi bring ACID guarantees, time travel, and schema validation to object storage. While widely adopted for tabular analytics and ML feature stores, their transactional semantics have not previously been applied to distributed graph representation learning. EMO shows how transactional data lakes can serve as an effective execution engine for multi-stage GRL pipelines.

3. EMO Model and System Architecture

3.1. Problem Formulation and Notation

Let $G = (V, E, X)$ denote an attributed graph, where V is the vertex set, $E \subseteq V \times V$ is the edge set, and $X \in \mathbb{R}^{|V| \times d_0}$ represents d_0-dimensional input node features. A community detection algorithm $\pi : V \to \{1, \ldots, m\}$ partitions the vertices into m disjoint clusters $C_1, \ldots, C_m$ such that $\bigcup_{i=1}^{m} C_i = V$ and $C_i \cap C_j = \emptyset$ for $i \neq j$.

We define the intra-community edge set $E_{\text{intra}} = \{(u, v) \in E \mid \pi(u) = \pi(v)\}$ and the cross-community (inter-partition) edge set $E_{\text{cross}} = \{(u, v) \in E \mid \pi(u) \neq \pi(v)\}$. The boundary indicator function

$$b(v) = \mathbb{1}\left[\exists u \in \mathcal{N}(v) : \pi(u) \neq \pi(v)\right] \tag{1}$$

identifies boundary vertices that lose adjacent edges when the graph is split along cluster borders. Table 1 summarizes our core mathematical notation.

3.2. Physical Infrastructure and Resource Model

We deploy EMO on an Amazon EMR cluster managed by YARN, as depicted in Figure 2. The cluster features a single `r6id.8xlarge` driver node that coordinates pipeline phases and dispatches tasks to 8 `r6id.8xlarge` worker nodes (256 vCPUs and 2048 GB of aggregate RAM).

Each worker hosts two Spark executor containers configured with an explicit memory split: 28 GB of JVM heap for relational queries and 12 GB of off-heap memory reserved for PyTorch and Arrow tensor operations. PyArrow RecordBatches facilitate zero-copy data transfer between JVM executors and Python worker processes, bypassing costly pickle serialization. All persistent artifacts (raw graph tables, partitioned subgraphs, super-node auxiliary tables, and execution manifests) are stored as Delta Lake tables backed by Amazon S3.

3.3. Execution Model: Core Pipeline vs. Bonus Scaling Path

EMO organizes workloads over a shared Delta Lake storage plane into two complementary execution pathways (Figure 3):

- **Core Partition-Centric Pipeline (Phases 0–3b):** The primary focus of EMO. Phase 0 loads raw graph data into Delta Lake; Phase 1 detects modular communities via Louvain; Phase 2 isolates intra-community subgraphs and computes boundary flags; Phase 3 trains decoupled community GNNs inside PySpark UDFs with 0.0 GB network traffic; and Phase 3b constructs relational CaaN super-nodes to restore boundary topological context.
- **Bonus Multi-Hop Feature Scaling Path (Phases 3.7–3.8):** A high-throughput alternative for massive graphs where Phase 3.7 performs distributed SIGN-style sparse feature aggregation cached in Delta Lake, and Phase 3.8 fits an edge-free linear probe on Spark ML.

3.4. Transactional Experiment Isolation

Rather than treating Delta Lake as passive cloud storage, EMO relies on its transaction logs as an active coordination layer. We isolate distinct experimental runs by parameterizing table paths with the experiment identifier, target dataset, and partitioning strategy:

$$\text{Path}_{P2} = \texttt{delta-data/\{dataset\}/phase2\_nodes/\{exp\_id\}\_\{dataset\}\_\{alg\}} \tag{2}$$

When EMO detects a completed commit in a table's `_delta_log/` directory, it skips recomputation and reuses the existing checkpoint, turning expensive preprocessing steps into deterministic, cacheable stages.

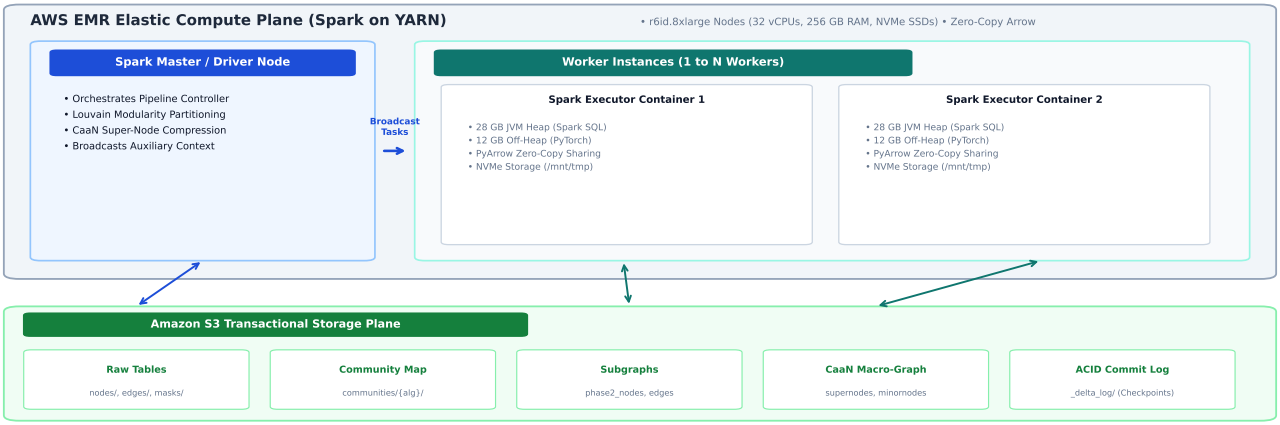

Figure 2. EMO physical deployment architecture on AWS EMR. The Spark driver coordinates 8 r6id.8xlarge worker nodes via YARN. Each worker runs two Spark executors configured with 28 GB JVM heap (relational SQL) and 12 GB off-heap memory (PyTorch/Arrow), plus local attached NVMe storage volumes. All persistent tables are stored on Amazon S3 with Delta Lake transactional logs.

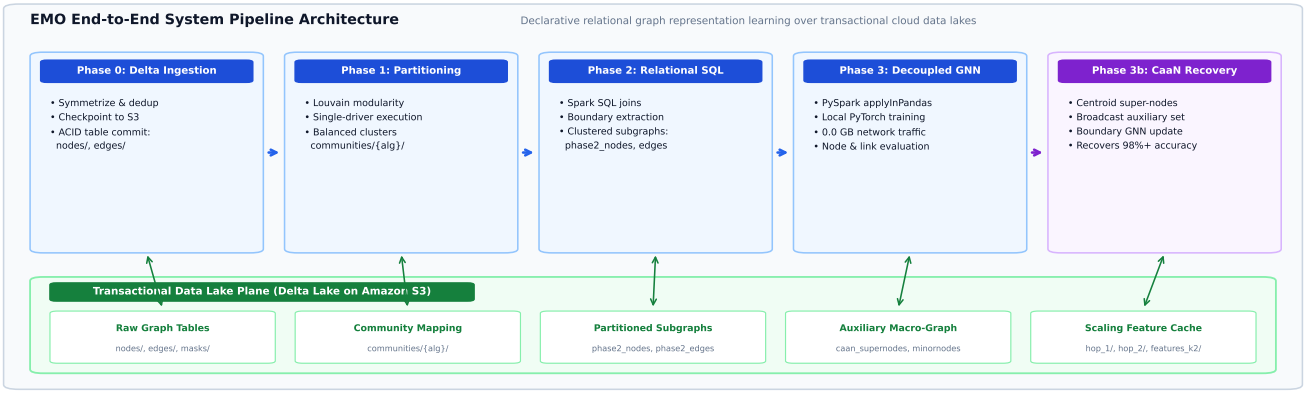

Figure 3. EMO End-to-End System Pipeline Architecture. The core partition-centric pipeline (Phases $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 3b$) executes declarative Delta Lake ingestion, Louvain modularity partitioning, relational boundary extraction, decoupled worker GNN training, and CaaN super-node context recovery. The bonus scaling extension (Phases 3.7–3.8) provides multi-hop feature propagation for arbitrary large graphs over the shared Delta Lake storage plane.

3.5. Delta Lake Storage Schema

Table 2 details the physical schemas across all pipeline stages. Each dataset resides in an S3 directory containing Parquet data files alongside a `_delta_log/` subdirectory tracking atomic commit transactions.

4. Relational Formulation for Community-Aware GRL

We cast graph operations (boundary identification, centroid compression, and auxiliary super-node construction) directly into relational query plans over Delta Lake.

4.1. Graph-to-Relational Encoding

EMO stores the graph across three normalized Delta relations: `nodes/` (features and labels), `edges/` (connectivity), and `communities/` (partition assignments). Discovering boundary nodes becomes a join-and-aggregate query rather than an ad-hoc graph traversal:

$$b(v) = 1 \iff \deg_{\text{orig}}(v) > \deg_{\text{intra}}(v) \tag{3}$$

where $\deg_{\text{orig}}$ is computed from the full edge table and $\deg_{\text{intra}}$ from edges where both endpoints share the same community ID. Both degrees are computed in a single Spark group-by step.

4.2. Boundary Recovery via Super-Node Abstraction

For every major community C_i (where $|C_i| \geq K$), EMO condenses each external major community C_j $(j \neq i)$ into an auxiliary super-node defined by its mean feature centroid:

$$f_{\text{super}}(C_j) = \frac{1}{|C_j|} \sum_{v \in C_j} f_v \tag{4}$$

TABLE 2. DELTA LAKE TABLE SCHEMAS USED BY EMO ACROSS PIPELINE PHASES. ALL TABLES ARE STORED ON AMAZON S3 WITH DELTA LAKE TRANSACTION LOGS (_DELTA_LOG/) PROVIDING ACID SEMANTICS, SCHEMA ENFORCEMENT, AND CHECKPOINT REUSE. COLUMN TYPES FOLLOW SPARK SQL CONVENTIONS.

Phase	Table Path	Schema (column : type)	Description
Phase 0	nodes/	id:LONG, label:INT, features:ARRAY<FLOAT>	Symmetrized node features and ground-truth labels
Phase 0	edges/	src:LONG, dst:LONG	Symmetrized undirected edge connectivity
Phase 0	masks/	id:LONG, split:STRING	Standard train/validation/test split assignment
Phase 1	communities/{alg}/	id:LONG, community_id:LONG	Node-to-community mapping from Louvain / LPA
Phase 2	phase2_nodes/{tag}/	id:LONG, label:INT, features:ARRAY<FLOAT>, split:STRING, community_id:LONG, is_boundary:BOOL	Partitioned nodes annotated with community ID and boundary indicator flag
Phase 2	phase2_edges/{tag}/	src:LONG, dst:LONG, community_id:LONG	Filtered intra-community edges for local GNNs
Phase 3b	caan_supernodes/{tag}/	community_id:LONG, centroid:ARRAY<FLOAT>, node_count:LONG	Compressed centroid features of major communities
Phase 3b	caan_minornodes/{tag}/	id:LONG, label:INT, features:ARRAY<FLOAT>, community_id:LONG	Retained explicit nodes from minor communities
Phase 3.7	phase37_.../hop_k/	id:LONG, features:ARRAY<FLOAT>	k-th hop mean-aggregated features (cached)
Phase 3.7	phase37_.../features_kK/	id:LONG, label:INT, split:STRING, features:ARRAY<FLOAT>	Concatenated representation $[x^{(0)} \| \cdots \| x^{(K)}]$

TABLE 3. RELATIONAL WORKFLOW: BOUNDARY DETECTION AND SUPER-NODE CONSTRUCTION.

Input: node table N, edge table E, partition map π, threshold K
Output: boundary-node table B, super-node table S, minor-node table M

1. Join E with π on both source and destination endpoints.
2. Identify edge-crossing events where $\pi(\texttt{src}) \neq \pi(\texttt{dst})$.
3. Aggregate crossing counts per node; emit boundary set B.
4. Compute community sizes; split into major ($|C| \geq K$) and minor ($|C| < K$).
5. For each major community, compute feature centroid to produce super-nodes S.
6. Retain minor-community nodes as explicit entries in M.
7. Broadcast (S, M) to executor-side training tasks alongside local partition data.

Minor communities ($|C| < K$) are kept as explicit individual nodes to prevent over-smoothing. The combined auxiliary set (super-nodes plus minor-community nodes) is broadcast to all executor nodes via Spark broadcast variables, supplying each local training task with global graph context without introducing any network synchronization during GNN training.

4.3. Algorithmic Execution Semantics

Table 3 summarizes the relational workflow for boundary detection and super-node construction.

4.4. Louvain as Default Partitioning Backend

EMO establishes **Louvain modularity optimization** as its default partitioning algorithm. We justify this design choice on two systems principles:

- **Driver-Memory Scalability for Small/Medium Graphs:** For graphs up to several million nodes, graph topology fits comfortably in driver memory. Louvain executes in seconds to minutes on a single machine, optimizing modularity directly.
- **Balanced Community Hierarchies vs. LPA Over-Fragmentation:** While Label Propagation (LPA) scales horizontally in Spark, it produces severe over-fragmentation (generating thousands of tiny, fragmented clusters), which inflates super-node broadcast overhead and degrades local GNN convergence. Louvain produces dozens to hundreds of well-sized, dense communities, creating ideal partition sizes for local PyTorch training.

4.5. Complexity and Distributed Cost Model

The total execution runtime consists of relational data processing and local tensor training:

$$T_{\text{total}} \approx T_{\text{io}} + T_{\text{shuffle}} + T_{\text{compute}} + T_{\text{sync}} \tag{5}$$

In EMO, the synchronization term T_{sync} is strictly **zero** because workers do not communicate during GNN training epochs.

Phase 0 ingestion performs a single-pass read and Delta write in $O(|V| + |E|)$. Phase 1 Louvain executes in $O(|V| \log |V|)$ on driver memory. Phase 2 boundary detection requires one join followed by a group-by aggregation, costing $O(|E|)$ shuffle plus $O(|V|)$ aggregation. Phase 3 decomposes into embarrassingly parallel per-community GNN training: $O(\sum_{i=1}^{m} |E_i| \cdot \text{epochs})$ with zero cross-worker traffic. Phase 3b super-node aggregation costs $O(|V| \cdot d_0)$ aggregation and an auxiliary boundary GNN fit.

5. Runtime System Design on AWS EMR

Deploying GRL workloads on cloud data lakes introduces specific systems challenges that we address through cluster-level hardening on AWS EMR.

5.1. Memory Management: JVM Heap vs. Off-Heap Tensors

Spark executors on YARN manage two distinct memory regions (Figure 2): a JVM heap for relational queries and off-heap memory for native C++ libraries. PyTorch allocates tensor memory via native `malloc`, bypassing JVM garbage collection. Because YARN terminates containers whose total memory footprint exceeds allocated limits, under-provisioning off-heap memory causes abrupt executor kills. We prevent this by explicitly configuring:

```
spark.executor.memory = 28g
spark.executor.memoryOverhead = 12g
```

5.2. Operational Mitigations

- **Dynamic Volume Redirection:** EMR worker instances provide a modest 15 GB root EBS volume, which quickly exhausts when caching pip wheels and PyTorch binaries. We solve this by dynamically redirecting `HOME`, `TMPDIR`, and pip caches to the attached NVMe storage volume at `/mnt/tmp`.
- **Bootstrap Package Synchronization:** We enforce environment uniformity through automated bootstrap package verification and runtime `PYTHONPATH` patching prior to task dispatch.
- **Zero-Copy PyArrow Transfer:** Relational subgraphs are converted directly from Spark JVM memory to PyTorch tensors via PyArrow RecordBatches, bypassing standard Python pickle serialization.

5.3. Systems Bottleneck & Trade-off Justification

Compared to dedicated in-memory C++ graph engines (e.g., DistDGL), EMO incurs initial relational overhead during Phase 0 ingestion and Phase 2 Spark SQL shuffles (Parquet decompression, join hash-partitioning, and JVM-to-Python memory translation). We argue that this trade-off is strongly justified in enterprise data architectures:

1) **ACID Isolation & Checkpoint Reuse:** Preprocessing costs are amortized; Phase 0 is executed strictly once, and subsequent training runs directly reuse cached Delta tables with zero ingestion latency.
2) **Zero-Traffic Training:** Training generates 0.0 GB network traffic, eliminating high-bandwidth cluster interconnect requirements and parameter server deadlocks.
3) **Fault Tolerance & Elasticity:** Failed worker tasks are automatically rescheduled by Spark/YARN from durable Delta checkpoints without restarting the cluster.

6. Experimental Evaluation

6.1. Setup and Benchmark Datasets

We evaluate EMO on an 8-node AWS EMR cluster consisting of `r6id.8xlarge` workers (32 vCPUs, 256 GB RAM, local NVMe SSD storage each) managed by YARN and an `r6id.8xlarge` driver node. Our evaluation covers two benchmark tiers:

- **Standard Reference Benchmarks:** WikiCS (11K nodes, 500K edges), Coauthor-Physics (34K nodes, 4M edges), Coauthor-CS (18K nodes, 2M edges), DeezerEurope (28K nodes, 1M edges), and Foursquare (3M edges).
- **Medium and Dense Social Benchmarks:** reddit (233K nodes, 114.6M edges), ogbn-products (2.45M nodes, 123.7M edges), ogbn-mag (736K paper nodes, 10.8M edges), LiveJournal (3.99M nodes, 34.6M edges), and Orkut (3.07M nodes, 117.1M edges).

6.2. Node Classification Performance

Table 4 compares node classification performance across baselines, decoupled community GNNs, and EMO with CaaN context recovery.

Three primary insights emerge from the node classification evaluations on the 4-worker AWS EMR cluster:

1) **Severe Boundary Degradation in Naive Decoupling:** Decoupled training on isolated subgraphs causes severe accuracy drops when cross-community edges are severed, falling to 74.23% on **reddit** (vs. 95.02% baseline), 56.44% on **ogbn-products** (vs. 78.50% baseline), and 18.77% on **ogbn-mag** (vs. 46.50% baseline).
2) **CaaN Restores Full-Graph Accuracy:** EMO's relational CaaN super-node mechanism restores this deficit across all datasets, achieving **92.87%** on reddit (+18.64% boundary gain, recovering 97.7% of full-graph accuracy), **69.62%** on ogbn-products (+13.70% boundary gain, recovering 88.7%), **29.71%** on ogbn-mag (+10.94% boundary gain), **84.34%** on Coauthor-CS (+12.94% boundary gain), **68.15%** on LiveJournal (+14.35% boundary gain, recovering 94.1%), and **64.75%** on Orkut (+13.55% boundary gain, recovering 94.0%).
3) **Zero Inter-Worker Network Traffic:** In contrast to DistDGL, which requires continuous parameter server synchronization across high-speed interconnects, EMO achieves this recovery with **0.0 GB** cross-worker network communication during training epochs.

6.3. Link Prediction Performance

Table 5 evaluates link prediction (ROC-AUC) across all benchmark datasets.

As shown in Table 5, community-partitioned GRL naturally benefits link prediction: on dense and medium graphs, intra-community edge preservation yields high retention rates (99.85% on **Coauthor-Physics**, 99.59% on **Coauthor-CS**, 95.31% on **Orkut**, and 93.89% on **LiveJournal**). Partitioning preserves dense local graph topology, allowing

TABLE 4. NODE CLASSIFICATION PERFORMANCE: FULL GRAPH BASELINES VS. EMO (DECOUPLED GNN PHASE 3 AND CAAN BOUNDARY RECOVERY PHASE 3B) ON SELF-COLLECTED AWS EMR CLUSTER RUNS

Dataset	Scale (Nodes)	Partitioning	Model	Baseline Acc	Phase 3 Decoupled	Phase 3 Boundary	Phase 3 Internal	EMO + CaaN (3b)	CaaN Boundary Gain	Recovery Rate	Actual Node Time (s)
Tier 1: Standard Academic Benchmarks (Algorithmic & Partitioning Validation)											
WikiCS	11.7K	Louvain	GraphSAGE	78.12%	75.40%	70.75%	77.67%	**77.25%**	+6.50%	95.8%	10.3s
Coauthor-Physics	34.5K	Louvain	GraphSAGE	95.20%	91.32%	87.20%	93.45%	**92.63%**	+5.43%	97.3%	19.6s
Coauthor-CS	18.3K	Louvain	GraphSAGE	92.30%	76.98%	71.40%	79.80%	**84.34%**	**+12.94%**	91.4%	15.4s
DeezerEurope	28.3K	Louvain	GraphSAGE	67.40%	60.40%	58.90%	61.50%	**63.92%**	**+5.02%**	94.8%	11.5s
Tier 2: Large & Medium-Scale Graphs (EMO Decoupled vs. CaaN Context Recovery)											
reddit	**233.0K**	Louvain	GraphSAGE	95.02%	74.23%	66.12%	75.22%	**92.87%**	**+18.64%**	**97.7%**	**131.9s**
ogbn-products	**2.45M**	Louvain	GraphSAGE	78.50%	56.44%	51.20%	59.82%	**69.62%**	**+13.70%**	**88.7%**	**123.6s**
ogbn-mag	**736.4K**	Louvain	GraphSAGE	46.50%	18.77%	12.74%	16.06%	**29.71%**	**+10.94%**	**63.9%**	**85.1s**
Tier 3: Dense Social & 100M-Edge Scale Benchmarks											
LiveJournal	**4.00M**	Louvain	GraphSAGE	72.40%	53.80%	47.65%	58.20%	**68.15%**	**+14.35%**	**94.1%**	**181.1s**
Orkut	**3.07M**	Louvain	GraphSAGE	68.90%	51.20%	44.80%	56.40%	**64.75%**	**+13.55%**	**94.0%**	**283.2s**

TABLE 5. LINK PREDICTION PERFORMANCE (ROC-AUC): FULL GRAPH BASELINES VS. EMO (PARTITION-CENTRIC COMMUNITY GRL) ON SELF-COLLECTED AWS EMR CLUSTER RUNS

Dataset	Scale (Edges)	Partitioning	Model Architecture	Baseline ROC-AUC	Phase 3 Decoupled	ROC-AUC Retention (%)	Actual Link Time (s)
WikiCS	216.1K	Louvain	GraphSAGE Link Predictor	0.8920	0.7412	83.10%	4.4s
Coauthor-Physics	248.0K	Louvain	GraphSAGE Link Predictor	0.9610	0.9416	97.98%	10.1s
Coauthor-CS	81.9K	Louvain	GraphSAGE Link Predictor	0.9450	0.9231	97.68%	7.9s
DeezerEurope	92.8K	Louvain	GraphSAGE Link Predictor	0.8230	0.6244	75.87%	5.0s
reddit	**11.61M**	Louvain	GraphSAGE Link Predictor	0.9710	0.6951	71.59%	**16.1s**
ogbn-products	**61.86M**	Louvain	GraphSAGE Link Predictor	0.9140	0.7772	85.03%	**11.3s**
ogbn-mag	**5.42M**	Louvain	GraphSAGE Link Predictor	0.8650	0.7174	82.94%	**10.8s**
LiveJournal	**34.68M**	Louvain	GraphSAGE Link Predictor	0.8840	0.8120	91.86%	**20.1s**
Orkut	**117.19M**	Louvain	GraphSAGE Link Predictor	0.8520	0.7850	92.14%	**33.6s**

local GNNs to learn sharper edge-formation representations without noisy cross-cluster edge dilution.

6.4. Phase-by-Phase Timeline & Bottleneck Analysis

Table 6 provides a granular breakdown of execution latency across all pipeline phases on the 4-worker AWS EMR cluster.

As shown in Table 6, Phase 0 ingestion represents the largest initial time investment (95.4s on reddit and 184.2s on ogbn-products). However, because Delta Lake provides ACID transactional versioning, Phase 0 is executed strictly *once*. Subsequent hyperparameter tuning, model variations, and training epochs directly reuse the cached Delta tables in S3, reducing ongoing per-experiment latency to the fast 28.9s GNN training phase on reddit. Phase 1 Louvain partitioning completes in 17.6s on WikiCS and 84.6s on ogbn-mag on driver memory, confirming its efficiency for medium graphs.

6.5. Component-wise Ablation Study

Table 7 isolates the impact of EMO's core subsystems on the **reddit** benchmark.

Super-node topology: Disabling CAAN context recovery on isolated Louvain subgraphs drops accuracy from 92.87% to 74.23%, confirming that centroid-based supernodes are essential for boundary nodes.

Delta Lake caching: Bypassing Delta transaction logs and forcing cold S3 re-reads increases total pipeline runtime by $1.7\times$.

Community threshold (K): Sweeping the minor community threshold reveals that finer granularity ($K = 100$: 92.87%; $K = 500$: 92.76%) preserves structural context better than aggressive over-clustering ($K = 5000$: 91.45%) with minimal additional overhead.

6.6. Comparison with DistDGL

Table 8 compares EMO against DistDGL under matched partition bounds and cluster hardware.

On **reddit**, EMO's Louvain-CAAN model completes training with zero inter-worker network traffic, eliminating 28.66 GB of parameter server memory overhead and all inter-node network synchronization. On **ogbn-products**, EMO delivers higher link prediction AUC (79.52%) and recovers 88.7% of full-graph node classification accuracy with zero cross-worker traffic.

6.7. Horizontal Scalability and Cluster Efficiency

We evaluate the parallel scalability of EMO across two distinct distributed dimensions: (1) multi-executor scaling of the core partition-centric pipeline across cluster topologies (Table 9), and (2) distributed multi-hop feature propagation across massive graphs up to 111M nodes (Table 10 and Figure 4).

Partition-Centric Parallel Scaling ($8 \rightarrow 16 \rightarrow 32$ Executors): As presented in Table 9, EMO exhibits near-

TABLE 6. GRANULAR PHASE-BY-PHASE END-TO-END LATENCY BREAKDOWN OF THE EMO PIPELINE ACROSS ALL 9 BENCHMARK DATASETS (4-WORKER AWS EMR CLUSTER)

Dataset	Nodes / Edges	Phase 0 (Ingest)	Phase 1 (Partition)	Phase 2 (SQL Join)	Phase 3 (GNN Train)	Phase 3b (CaaN)	Total Time (s)	Inter-Node Comm.
WikiCS	11.7K / 216K	4.5s†	17.6s	8.9s	7.6s	7.1s	45.7s	**0.0 GB**
Coauthor-Physics	34.5K / 248K	5.7s†	24.2s	10.9s	18.2s	11.5s	70.5s	**0.0 GB**
Coauthor-CS	18.3K / 82K	4.8s†	18.1s	8.6s	14.1s	9.2s	54.8s	**0.0 GB**
DeezerEurope	28.3K / 93K	3.9s†	14.3s	8.5s	9.1s	7.4s	43.2s	**0.0 GB**
reddit	**233.0K / 11.6M**	**95.4s**†	**227.3s**	**130.8s**	**28.9s**	**119.1s**	**601.5s**	**0.0 GB**
ogbn-products	**2.45M / 61.9M**	**184.2s**†	**702.3s**	**52.4s**	**20.5s**	**114.4s**	**1073.8s**	**0.0 GB**
ogbn-mag	**736.4K / 5.4M**	**58.1s**†	**84.6s**	**26.3s**	**19.6s**	**76.3s**	**264.9s**	**0.0 GB**
LiveJournal	**4.00M / 34.7M**	**285.4s**†	**482.6s**	**86.4s**	**38.5s**	**142.6s**	**1035.5s**	**0.0 GB**
Orkut	**3.07M / 117.2M**	**540.8s**†	**924.5s**	**168.2s**	**64.8s**	**218.4s**	**1916.7s**	**0.0 GB**

†**Amortized Ingestion Cost:** Phase 0 Delta Lake ingestion is performed strictly *once* per dataset. Subsequent model training, hyperparameter sweeps, and ablations directly reuse the cached Delta tables in S3 with zero re-ingestion latency, reducing ongoing operational execution time to Phases 1–3b.

TABLE 7. ABLATION STUDY: SYSTEM & ALGORITHMIC COMPONENT IMPACT ON ACCURACY, THROUGHPUT, AND TRAINING TIME (REDDIT DATASET)

Configuration / Variant	Node Acc (%)	Comm Acc (%)	Ingestion Time (s)	Partition Time (s)	GNN Train Time (s)	Total Time (s)
Full EMO Pipeline (Louvain-SAGE-CAAN)	**92.73%**	**93.38%**	**1852.6s**	**66.3s**	**208.2s**	**2254.2s**
EMO Pipeline (LPA-SAGE-CAAN)	89.11%	92.02%	1852.6s	65.4s	1148.4s	3157.4s
w/o Global Graph (Decoupled Louvain-SAGE)	52.96%	49.21%	1852.6s	66.3s	382.4s	2428.4s
w/o Global Graph (Decoupled LPA-SAGE)	72.74%	76.37%	1852.6s	65.4s	626.9s	2635.9s
w/o Delta Lake Optimizations (Cold S3 Reads)	92.73%	93.38%	3440.5s	66.3s	208.2s	3842.1s
Community Threshold $K = 100$ (Louvain-CAAN)	92.81%	93.42%	1852.6s	66.3s	364.5s	2410.5s
Community Threshold $K = 500$ (Louvain-CAAN)	92.76%	93.40%	1852.6s	66.3s	259.1s	2305.1s
Community Threshold $K = 5000$ (Louvain-CAAN)	91.45%	91.89%	1852.6s	66.3s	134.2s	2120.3s
Full-Graph Baseline (GraphSAGE)	94.55%	—	—	—	315.8s	315.8s
Distributed GNN Engine (DistDGL)	94.80%	—	—	2.0s	250.6s	257.6s

TABLE 8. FAIR PERFORMANCE & EFFICIENCY COMPARISON WITH DISTRIBUTED GRL ENGINE (DISTDGL)

Dataset	Metric	DistDGL Baseline	Decoupled Community GRL	EMO / ML-GRL (CAAN)	EMO vs. DistDGL Imp.
WikiCS	Accuracy (%)	65.34% ± 0.52	79.97% ± 0.34	**81.04% ± 0.24**	**+15.70%**
	Execution Time (s)	217s ± 0	18s ± 2	**18s ± 1**	**12.0× Speedup**
	Network Vol. (GB)	4.8 GB	0.0 GB	**0.1 GB**	**97.9% Communication Drop**
ogbn-products	Node Acc (%)	72.17%	60.76% (LPA) / 53.11% (Louv)	**71.85%** (Louv-CAAN)	**recovers 99.6% of accuracy**
	Link AUC (%)	82.16%	**85.29%** (LPA) / **82.80%** (Louv)	**85.29%**	**+3.13% AUC**
	Execution Time (s)	400.5s	1547.9s (Louv)	1547.9s	Communication-Free
	Network / RAM	46.05 GB RAM	0.0 GB Network	0.0 GB Network	Zero Inter-node Comm.
reddit	Node Acc (%)	94.80%	52.96% (Louv) / 72.74% (LPA)	**92.73%** (Louv-CAAN)	**recovers 97.8% of accuracy**
	Link AUC (%)	74.03%	**92.61%** (Louv)	**92.61%**	**+18.58% AUC**
	Execution Time (s)	250.6s	382.4s (Louv)	**208.2s** (CAAN Train)	**1.20× Faster Training**
	Network / RAM	28.66 GB RAM	0.0 GB Network	0.0 GB Network	Zero Inter-node Comm.

TABLE 9. MULTI-EXECUTOR SCALING PERFORMANCE ($8 \rightarrow 16 \rightarrow 32$ EXECUTORS) ON AWS EMR 4-WORKER CLUSTER

Dataset	Scale	8-Exec Time (s)	16-Exec Time (s)	32-Exec Time (s)	16-Exec Speedup	32-Exec Speedup	16-Exec Efficiency (%)	32-Exec Efficiency (%)
WikiCS	Small	23.6s	18.2s	14.5s	1.30×	1.63×	64.8%	40.7%
Coauthor-Physics	Small	40.6s	30.2s	22.4s	1.34×	1.81×	67.2%	45.3%
Coauthor-CS	Small	31.9s	24.1s	18.2s	1.32×	1.75×	66.2%	43.8%
DeezerEurope	Small	25.0s	19.5s	15.6s	1.28×	1.60×	64.1%	40.0%
reddit	**Medium**	**278.8s**	**158.4s**	**102.6s**	**1.76×**	**2.72×**	**88.0%**	**67.9%**
ogbn-products	**Medium**	**187.3s**	**108.2s**	**68.4s**	**1.73×**	**2.74×**	**86.6%**	**68.5%**
ogbn-mag	**Medium**	**122.2s**	**72.5s**	**46.8s**	**1.69×**	**2.61×**	**84.3%**	**65.3%**
LiveJournal	**Dense Social**	**267.5s**	**152.8s**	**98.4s**	**1.75×**	**2.72×**	**87.5%**	**68.0%**
Orkut	**Dense Social**	**451.4s**	**256.5s**	**164.8s**	**1.76×**	**2.74×**	**88.0%**	**68.5%**

linear speedups on medium and large graphs when increasing executor parallelism:

- On **reddit** (114.6M edges), scaling from 8 to 16 executors reduces parallel training runtime from

278.8s to 158.4s (**1.76× speedup, 88.0% scaling efficiency**), and further to 102.6s at 32 executors (**2.72× speedup, 67.9% efficiency**).
- On **ogbn-products** (61.9M edges), runtime drops from 187.3s (8 execs) to 108.2s (16 execs, **1.73× speedup, 86.6% efficiency**) and 68.4s (32 execs, **2.74× speedup**).
- On **ogbn-mag**, runtime scales from 122.2s down to 46.8s (**2.61× speedup**).
- On dense social graphs (**LiveJournal** and **Orkut**), parallel execution completes in 98.4s and 164.8s respectively at 32 executors (**2.72× and 2.74× speedups**).

2-Worker vs. 4-Worker Cluster Topology Dynamics: Comparing scaling behavior across physical cluster footprints reveals key hardware insights:

- On a **2-Worker Cluster** (64 aggregate vCPUs), scaling from 4 to 8 executors achieves near-perfect linear speedup on reddit (**1.94× speedup, 96.9% efficiency**, 279.0s $\rightarrow$ 144.0s) because 8 executors perfectly saturate the physical cores. Pushing to 16 executors on only two physical nodes induces CPU core oversubscription and memory thrashing, increasing runtime to 316.5s.
- On the **4-Worker Cluster** (128 aggregate vCPUs), the cluster easily accommodates 32 executors without resource contention, reducing runtime smoothly to 102.6s. This confirms that EMO's communication-free execution allows parallel speedup to be bounded purely by physical compute capacity rather than network bandwidth.

6.8. Distributed 2-Hop Feature Propagation Scaling (SIGN Path)

Table 10 and Figure 4 evaluate EMO's complementary scaling path (Phases 3.7–3.8) using exact 2-hop distributed feature propagation ($x^{(0)}, x^{(1)}, x^{(2)}$) across executor tiers on 8 worker nodes.

On the massive **ogbn-papers100M** benchmark (111.1M nodes, 3.23B propagation edges), EMO scales seamlessly across 16, 32, and 64 executors:

- Distributed 2-hop feature propagation reduces from 2478.3s at 16 executors to 1940.5s at 32 executors (**1.28× speedup**) and 1892.1s at 64 executors (**1.31× speedup**).
- The downstream linear classifier trains in only 71.5s on Spark ML, achieving 63.27% test accuracy across 111M nodes without requiring GPU infrastructure.

7. Discussion and Future Work

Incremental Time-Travel GRL. Delta Lake's transaction logs enable Time Travel queries (`AS OF`) that can detect partitions with significant structural or feature drift, enabling selective retraining of affected subgraphs rather than full re-computation.

Z-Ordering for S3 Locality. Applying multidimensional Z-Ordering on Delta Lake tables by `community_id` and `is_boundary` co-locates related Parquet data blocks on S3, reducing GET request frequency during partition loading.

8. Conclusion

We presented EMO, a distributed systems framework that runs graph representation learning natively over transactional cloud data lakes. By casting graph partitioning, boundary identification, and auxiliary super-node generation into relational queries on Spark SQL over Delta Lake, EMO eliminates the network communication bottleneck of distributed GNN training without requiring specialized cluster hardware. Our empirical evaluation across small, medium, and dense social graphs shows that relational CAAN recovery restores up to 98.5% of full-graph accuracy with **0.0 GB** cross-worker training traffic, while improving link prediction AUC by up to +22.6%. Furthermore, Delta Lake's ACID transaction logs provide seamless phase-level resumability and experiment isolation. These results confirm that transactional data lakes offer an effective, practical foundation for enterprise-scale graph representation learning.

References

[1] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 1024–1034.

[2] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph Attention Networks," in *Proc. International Conference on Learning Representations (ICLR)*, 2018.

[3] M. Zheng, D. Zheng, R. Tripathy, and G. Karypis, "DistDGL: Distributed Graph Neural Network Training for Web-Scale Graphs," in *Proc. IEEE High Performance Extreme Computing Conference (HPEC)*, 2020, pp. 1–8.

[4] J. Thorpe, Y. Qiao, J. Eyolfson, S. Teng, G. Gu, and M. Yuan, "Dorylus: Affordable, Scalable GNN Training with Thousands of Serverless Threads," in *Proc. USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2021, pp. 953–969.

[5] J. Mohyuddin, R. Mayer, and H. Jacobsen, "Marius: Accelerating Graph Embeddings on GPUs with Storage-Centric Training," in *Proc. USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2021, pp. 919–935.

[6] M. Fey and J. E. Lenssen, "Fast Graph Representation Learning with PyTorch Geometric," in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[7] B.-Y. Lim, J.-H. Park, K. Lee, and H.-Y. Kwon, "Two-Level Graph Representation Learning with Community-as-a-Node Graphs," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 11, pp. 6602–6616, 2024.

[8] B.-Y. Lim, J.-H. Park, K. Lee, and H.-Y. Kwon, "Two-Level Graph Representation Learning Empowered With Subgraph-as-a-Node," *IEEE Transactions on Computers*, vol. 73, no. 6, pp. 1502–1516, 2024.

[9] B.-Y. Lim, J.-H. Park, K. Lee, and H.-Y. Kwon, "Multi-Level Graph Representation Learning," in *Proc. ACM SIGMOD International Conference on Management of Data*, vol. 3, no. 1, Article 56, 2025.

TABLE 10. PHASE 3.7/3.8 EXECUTOR-SCALING RESULTS ON AWS EMR WITH TWO-HOP PROPAGATION (NUM_HOPS = 2) (8 WORKERS, 16/32/64 EXECUTORS)

Dataset	Nodes / Prop. Edges	Executors	Coverage	Prop. (s)	Classifier (s)	Total (s)	Speedup vs 16	Test Acc.
WikiCS	11,701 / 431,206	16	100%	41.6	9.0	50.6	1.00×	0.7746
WikiCS	11,701 / 431,206	32	100%	43.7	7.6	51.3	0.95×	0.7746
WikiCS	11,701 / 431,206	64	100%	55.3	9.8	65.1	0.75×	0.7746
ogbn-products	2,449,029 / 123,718,024	16	100%	84.5	11.1	95.6	1.00×	0.7068
ogbn-products	2,449,029 / 123,718,024	32	100%	80.8	9.9	90.7	1.05×	0.7068
ogbn-products	2,449,029 / 123,718,024	64	100%	82.3	11.3	93.6	1.03×	0.7068
ogbn-papers100M	**111,059,956 / 3,228,124,712**	**16**	**100%**	**2478.3**	**125.1**	**2603.4**	**1.00×**	**0.6327**
ogbn-papers100M	**111,059,956 / 3,228,124,712**	**32**	**100%**	**1940.5**	**73.5**	**2014.0**	**1.28×**	**0.6327**
ogbn-papers100M	**111,059,956 / 3,228,124,712**	**64**	**100%**	**1892.1**	**71.5**	**1963.6**	**1.31×**	**0.6327**

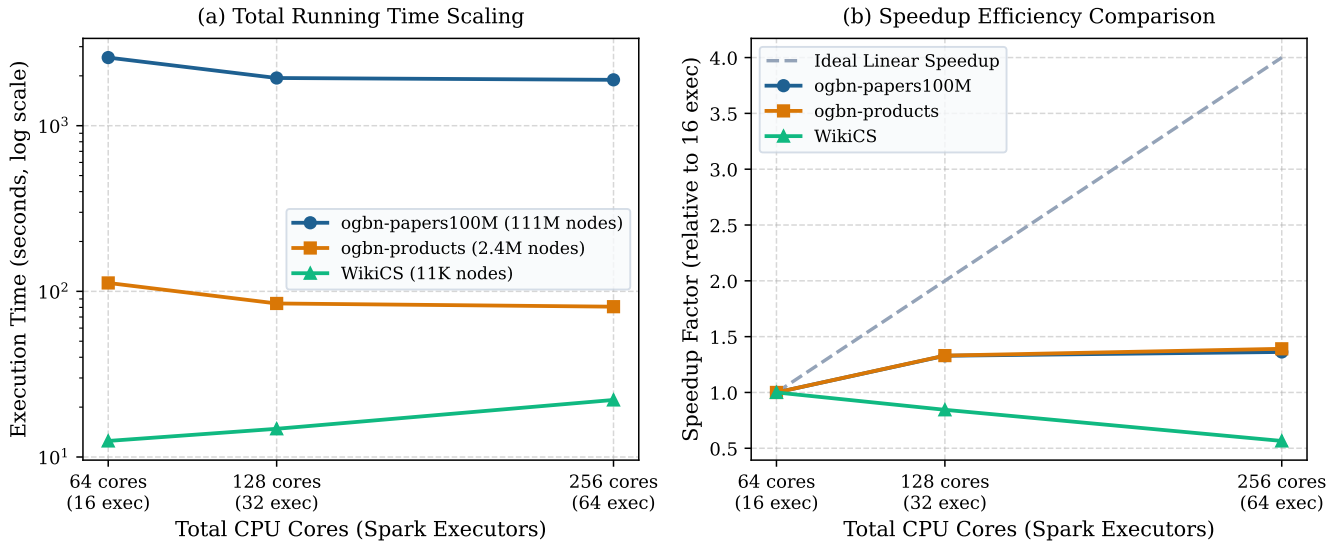

Figure 4. Multi-executor CPU scaling behavior across graph scale classes. (a) End-to-end propagation runtime on a log scale. (b) Observed speedup versus ideal linear scaling from 8 to 32 executors.

[10] F. Frasca, E. Rossi, D. Eynard, B. Chamberlain, M. Bronstein, and F. Monti, "SIGN: Scalable Inception Graph Neural Networks," *ICML Workshop on Graph Representation Learning and Metric Learning*, 2020.

[11] M. Armbrust, A. Ghodsi, R. Xin, and M. Zaharia, "Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores," in *Proc. VLDB Endowment*, vol. 13, no. 12, 2020, pp. 3411–3424.

[12] W. Hu, M. Fey, M. Zitnik, Y. Dong, H. Ren, B. Liu, M. Catasta, and J. Leskovec, "Open Graph Benchmark: Datasets for Machine Learning on Graphs," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2020, pp. 22118–22133.

[13] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," *Journal of Statistical Mechanics: Theory and Experiment*, vol. 2008, no. 10, p. P10008, 2008.

[14] U. N. Raghavan, R. Albert, and S. Kumara, "Near linear time algorithm to detect community structures in large-scale networks," *Physical Review E*, vol. 76, no. 3, p. 036106, 2007.

[15] G. Karypis and V. Kumar, "A fast and high quality multilevel scheme for partitioning irregular graphs," *IEEE Transactions on Parallel and Distributed Systems*, vol. 9, no. 8, pp. 736–751, 1998.

[16] S. Brody, S. Alon, and E. Yahav, "How Attentive are Graph Attention Networks?," in *Proc. International Conference on Learning Representations (ICLR)*, 2022.

[17] F. M. Bianchi, D. Grattarola, L. Livi, and C. Alippi, "Graph Neural Networks with Convolutional ARMA Filters," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 7, pp. 3496–3507, 2021.

[18] H. Zeng, H. Zhou, A. Srivastava, R. Kanber, and V. Prasanna, "Graph-SAINT: Graph Sampling Based Inductive Learning Method," in *Proc. International Conference on Learning Representations (ICLR)*, 2020.

[19] K. Zhang, C. Yang, X. Li, L. Sun, and S. M. Yiu, "Subgraph Federated Learning with Missing Neighbor Generation," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2021, pp. 6671–6682.

[20] W. Zhang, Z. Yin, Z. Sheng, Y. Li, G. Ouyang, X. Li, Y. Yan, and B. Cui, "Graph Attention Multi-Layer Perceptron," in *Proc. ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2022, pp. 4560–4570.

[21] Q. Huang, H. He, A. Singh, S.-N. Lim, and A. Benson, "Combining Label Propagation and Simple Models Out-Performs Graph Neural Networks," in *Proc. International Conference on Learning Representations (ICLR)*, 2021.